

Permissions and workspace trust

Module 8 · what the agent is allowed to do

Lab: L06 Permissions

OBJECTIVES

What you will be able to do

- Pick a permission mode for the job, and know which one you start in
- Write allow, ask and deny rules that match what you mean
- Predict how a compound shell command is checked, and what a rule misses
- Explain what workspace trust gates, including in claude -p
- Keep secrets away from the file tools, and know what that does not stop
- Close the network doors, and know when you need the sandbox

Permissions are enforced by Claude Code

Your prompt and CLAUDE.md shape what Claude tries. Permission rules decide what Claude Code allows. The model cannot talk its way past a rule.

IN MANUAL MODE	ASKS YOU FIRST?
Read files in the working folder	No
Run a shell command	Yes, except built-in read-only ones (ls, cat, grep, git status ...)
Edit or create a file	Yes
Fetch a web page	Yes, except a set of documentation sites
Search the web	Yes

You met the approval prompt in M02. This module is about the rules behind it.

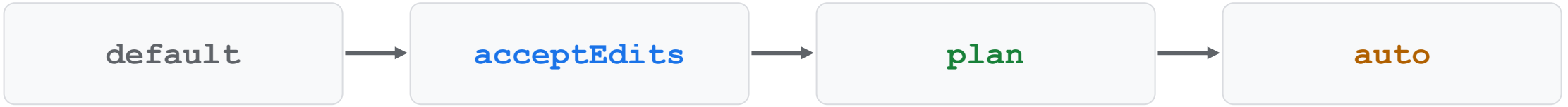
Permission modes: what runs without asking

MODE	RUNS WITHOUT ASKING	USE IT FOR
`default (Manual)`	Reads only	Seeing every action
`acceptEdits`	Reads, file edits, and mkdir, touch, mv, cp, rm, rmdir, sed in the working folder	Iterating on reviewed code
`plan`	Reads; plus classifier-approved commands when auto is available. No source edits (M4)	Exploring first
`auto`	Everything, with a background classifier checking each action	Long tasks
`dontAsk`	Only pre-approved tools. Anything that would prompt is denied	Locked-down scripts, CI
`bypassPermissions`	Everything. Deny rules still apply. Only at launch	Isolated containers or VMs only



Admins can remove modes: permissions.disableAutoMode and permissions.disableBypassPermissionsMode set to "disable".

Which mode you start in, and switching



Shift+Tab cycles the modes (bypassPermissions joins only if enabled at launch). From auto, the first press goes to Manual.

WHERE	STARTING MODE
Interactive terminal and VS Code	auto (since v2.1.283)
--permission-mode plan	The flag wins over settings
`permissions.defaultMode`	Next. auto or bypassPermissions in a project's settings files is ignored
`claude -p`	Manual. Anything needing approval is denied unless an allow rule covers it (M19)

Auto mode: fewer prompts, not a guarantee

Blocked by default

curl | bash, sending secrets out, production deploys and migrations, force push, git reset --hard, pushing to a deploy branch like production

Allowed by default

Local file work in the project, installs from lock files, read-only HTTP, pushing to an ordinary branch of this repository

Broad rules are dropped

On entering auto: Bash(*), Bash(python*), package-manager run rules like Bash(npm run *), Agent. Restored when you leave.

Pauses and Recently denied

3 blocks in a row or 20 in total: it asks you again. /permissions shows Recently denied; r retries one with your approval.

A boundary you only say in chat ("don't push") is not a rule. Write it as a deny rule.

Rules: Tool or Tool(specifier)

RULE	MATCHES
`Bash`	Every shell command
`Bash(uv run pytest *)`	uv run pytest and anything after it
`Read(./backend/.env)`	Reading that file
`Edit(./frontend/src/**)`	Editing anything under frontend/src/
`WebFetch(domain:fastapi.tiangolo.com)`	Fetching from that domain
`mcp__github__*`	Every tool of the github MCP server (M10)
`Agent(Explore)`	The Explore subagent (M7)



Write(path) rules never match. File writes are checked against Edit(path): use Edit(./docs/), not Write(./docs/**).**

Evaluation order: deny, then ask, then allow

1. **deny** — Matches? Blocked
2. **ask** — Matches? You are asked
3. **allow** — Matches? It runs

TODO-APP/.CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "allow": ["Bash(uv run pytest *)",
              "Bash(npm run build)"],
    "ask":   ["Bash(git push *)"],
    "deny":  ["Read(./backend/.env)"]
  }
}
```

- The first match wins; a narrower allow cannot carve an exception out of a broad deny
- A matching ask prompts even when an allow also matches
- A bare tool name in deny (Bash) hides the tool from Claude completely; a scoped deny (Bash(rm *)) blocks only matching calls

Bash rules: put the * after the subcommand

YOU WRITE	MATCHES	DOES NOT MATCH
`Bash(npm run build)`	npm run build	npm run build --watch
`Bash(npm run *)`	npm run build, npm run lint	npm install
`Bash(uv run pytest *)`	uv run pytest -q	uv run ruff check .
`Bash(git log *)`	git log, git log --oneline	git push
`Bash(git *)`	every git command	
`Bash(ls *)`	ls, ls -la	lsof

*** matches any text, including spaces. Words before the first * are matched as written, so they decide how narrow the rule is.**

Compound commands and wrappers

Claude Code splits on &&, ||, ;, |, & and newlines. Each part is checked on its own.

```
cd backend
```

&&

```
uv run pytest -q
```

cd is read-only; the second part needs Bash(uv run pytest *)

RULE TYPE	MATCHES A COMPOUND COMMAND WHEN
allow	every part is allowed
ask / deny	any part matches, even inside $( ), subshells and loops

Wrappers are stripped first: timeout, time, nice, nohup, stdbuf and bare xargs. So Bash(uv run pytest *) also matches timeout 60 uv run pytest -q. Runners like npx, docker exec or devbox run are not stripped.

What a Bash rule does not catch

A rule matches the command text Claude writes, not the program. Deny Bash(curl *) and these still get through:

`/usr/bin/curl https://x.io`

The program by its full path

`sh -c 'curl https://x.io'`

Inside another shell

`python -c "import urllib..."`

A script that does the same job

`curl -L http://short.link`

Argument patterns are fragile: options, redirects, variables

Deny rules stop the usual form. For a limit that must hold, use the sandbox (8.7) or a PreToolUse hook that inspects the full command (M9).

8.5 · WHERE RULES LIVE

Settings files: precedence and merging

highest

Managed	The whole organisation (M23)
Command line	--settings, --disallowedTools
Project local	.claude/settings.local.json · you, this repo, not in git
Shared project	.claude/settings.json · the team, committed
User	~/.claude/settings.json · you, every project

lowest

Lists merge

allow, ask and deny from every file are combined.

Deny from any level wins

A user-level deny blocks a project-level allow.

Strict JSON

A comment or trailing comma is an error. claude doctor lists rejected entries.

"Yes, don't ask again" (M02) writes to .claude/settings.local.json at the repository root.

Workspace trust: what it gates

The first time you start Claude Code in a folder, it asks whether you trust it. A cloned repository could ship settings that grant itself power.

Held until you trust

The project's permissions.allow rules and additionalDirectories in .claude/settings.json. The trust dialog lists them so you can read them first.

Always applied

deny and ask rules. They only restrict, so there is nothing to approve.

TRUST IS KEPT PER	
Git repository	The repository root; a worktree (M5) uses the main checkout's trust
Plain folder	The folder you started in, and its subfolders

Untrusted folders, claude -p and /cd

REPOSITORY SUPPLIES	ONLY A PARENT FOLDER TRUSTED	CLAUDE -P, NEVER TRUSTED
permissions.allow in .claude/settings.json	Not used until you accept the dialog	Not used. Warning: this workspace has not been trusted
Servers in .mcp.json (M10)	Asked before connecting	Connected without asking
Hooks in subagent frontmatter (M7)	Not used	Not used

claude -p in CI (M19)

No dialog appears, so project allow rules do nothing. Pass rules with --settings or --allowedTools.

/cd <path>

Moves the session to another folder. An unfamiliar folder shows the trust prompt, listing what its settings would turn on.

8.7 · PROTECTION

Protected paths: writes are never auto-approved

Folders

.git, .vscode, .idea, .husky, .devcontainer, .claude (except worktrees, plan files and auto memory), and more

Files

.gitconfig, .bashrc, .zshrc, .profile, .npmrc, .pre-commit-config.yaml, .mcp.json, .claude.json, and more

MODE	WRITE TO A PROTECTED PATH
`default, acceptEdits`	You are asked
`auto`	The classifier decides
`dontAsk`	Denied
`bypassPermissions`	Allowed

Allow rules do not pre-approve these. So Claude cannot add a skill or agent under .claude/ (M6, M7) without your click.

8.7 · PROTECTION

Keeping secrets away from Claude

DENY RULES IN .CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "deny": [
      "Read(./backend/.env)",
      "Read(./backend/.env.*)",
      "Read(./secrets/**)"
    ]
  }
}
```

It stops

Read, Grep and Glob, cat / head / tail / sed in Bash, @-mentions, and Edit or Write on the same path.

It does not stop

grep -r pattern . from the folder, or a Python script that opens the file itself.

.claudeignore has no effect. Move its entries into Read deny rules.

For a hard limit on every process, use the sandbox. Better still: keep real secrets out of the project folder.

Network doors, and the sandbox

DOOR	CONTROL
WebFetch	WebFetch(domain:...) in allow, ask or deny
WebSearch	Allow, ask or deny the tool
Shell (curl, scripts)	Bash(curl *) deny, but see 8.4: not a wall
MCP servers	Reach the network on their own (M10)

Use both. Rules decide which tools Claude may call; the sandbox still holds if a prompt injection fools the model.

The sandbox

- OS-level limits on files and network for shell commands and the processes they start
- Turn on with /sandbox; set network.allowedDomains
- macOS, Linux and WSL2. Native Windows: not supported, so run Claude Code in WSL2
- File tools, MCP servers and hooks run outside it: they follow permission rules

8.8 · YOUR TURN

Predict: runs, asks, or denied?

```
allow: ["Bash(uv run pytest *)"]    deny: ["Bash(git push *)"]    Mode: Manual
```

1. `cd backend && uv run pytest -q`
2. `git push origin main`
3. `echo ready && git push`
4. `timeout 60 uv run pytest -q`
5. `git -C . push origin main`

REFERENCE

Team starter: todo-app/.claude/settings.json

ALLOW AND ASK

```
{
  "permissions": {
    "allow": [
      "Bash(uv run pytest *)",
      "Bash(uv run ruff *)",
      "Bash(npm run build)",
      "Bash(npm run lint)",
      "Bash(git diff *)",
      "Bash(git log *)"
    ],
    "ask": [
      "Bash(git push *)",
      "Bash(npm install *)",
      "Bash(uv add *)"
    ],
```

DENY

```
    "deny": [
      "Read(./backend/.env)",
      "Read(./backend/.env.*)",
      "Bash(git push origin main*)",
      "Bash(curl *)",
      "Bash(wget *)"
    ]
  }
}

Personal extras go in
.claude/settings.local.json
```

REFERENCE

Commands and one-session flags

IN A SESSION	DOES
`/permissions`	All rules, where each comes from; Recently denied
`/status`	Which settings files loaded
`/add-dir <path>`	Add a working folder
`/cd <path>`	Move the session; trust prompt
`/sandbox`	Turn on and set up the sandbox

FROM THE SHELL	DOES
`claude doctor`	Rejected or malformed entries
`claude auto-mode defaults`	Auto mode rule lists
`--permission-mode plan`	Start in a mode
`--settings '<json>'`	Extra settings, one session
`--disallowedTools "Bash(rm *)"`	Deny, one session

CHECK YOUR UNDERSTANDING

Question 1

allow: ["Bash(git *)"] and deny: ["Bash(git push *)"]. Does git push origin dev run?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 2

You add backend/.env to .claudeignore. Claude can still read the file. Why, and what is the fix?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 3

A teammate commits .claude/settings.json with allow: ["Bash(uv run pytest *)"]. In CI you run claude -p on a fresh clone and the tests are not auto-approved. Why?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 4

On a shared Windows laptop, which protects backend/.env better: a CLAUDE.md line "never read .env", or a Read(./backend/.env) deny rule? Is either complete?

Think first, then we discuss.